

FloodSim GIS

Flood Simulation & Terrain Analysis System

v1.0

Qt6 / C++

OpenGL 3.3

Azure AI

 2D/3D Maps  Azure Cloud  AI Alerts  Flood Engine  Satellite Tiles

Platform	Language	AI Engine	Cloud
Windows 10/11	C++ / Qt6	LLaMA 3.1	Azure Blob

A professional-grade GIS flood simulation application combining real-world DEM data, HD satellite imagery, machine learning flood prediction, OpenGL 3D rendering, AI-generated alerts, and Microsoft Azure cloud integration into a single desktop application.

Contents

1	System Overview	3
1.1	What is FloodSim GIS?	3
1.1.1	Key Highlights	3
1.2	Technology Stack	4
1.3	Architecture Overview	4
2	File Structure	5
2.1	Project Layout	6
2.2	Module Responsibilities	7
3	Completed Features	8
3.1	DEM Loading & Generation	8
3.2	Satellite Imagery	8
3.3	Flood Simulation Engine	9
3.4	3D Terrain Rendering	9
3.5	Risk Marker System	10
3.6	2D Map Features	10
3.6.1	Flood Overlay	10
3.6.2	Contour Lines	11
3.6.3	Watershed Detection	11
3.6.4	Cell Click	11
3.7	AI-Powered Analysis	11
3.8	Azure Cloud Integration	12
3.9	Rainfall Data System	12
3.10	Flood Area Calculator	12
4	Environment Variables	13
5	Microsoft Store Release Checklist	14
5.1	What Needs to be Done	14
5.1.1	Technical Requirements	14
5.1.2	Core Features Still Missing	15
5.1.3	Polish & UX Tasks	15
5.2	Recommended Release Timeline	16
6	Data Flow Diagram	17
7	Future Roadmap	18
7.1	Time-Based Flood Simulation (Next Feature)	18

System Overview

1.1 What is FloodSim GIS?

i Project Summary

FloodSim GIS is a professional desktop GIS application for flood simulation and terrain analysis. Built entirely using **C++ with Qt6**, it combines real-world elevation data, HD satellite imagery, ML-based flood prediction, and advanced OpenGL 3D rendering into a single cohesive tool for disaster management professionals and researchers.

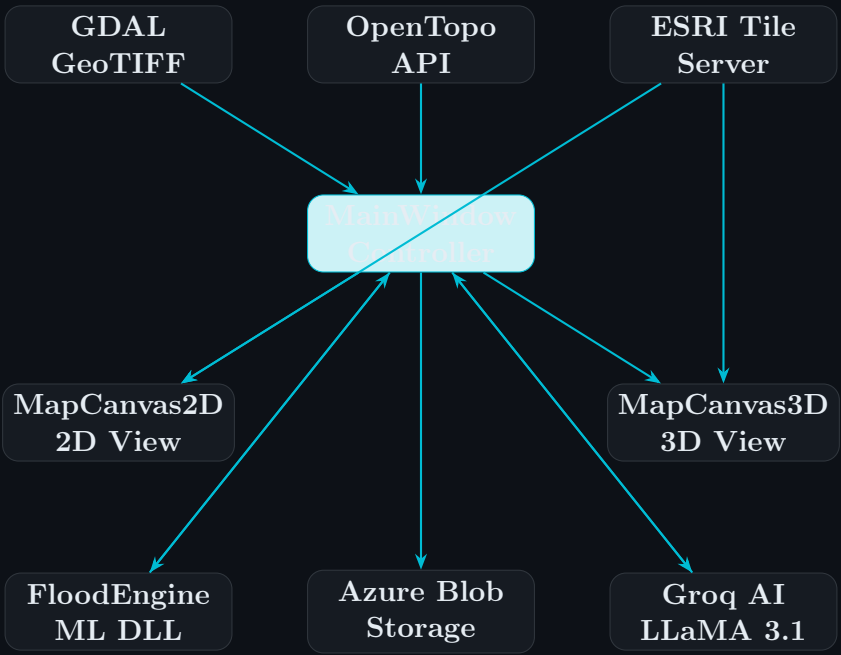
1.1.1 Key Highlights

- Real-time flood simulation via a custom Machine Learning DLL engine
- Multi-tile HD satellite imagery draping over 3D terrain mesh
- Synchronized 2D and 3D map views with shared data layer
- AI-generated flood alerts and technical impact reports (Groq LLaMA 3.1)
- Microsoft Azure Blob Storage integration for all simulation data
- Watershed boundary detection using D8 flow direction algorithm
- Topographic contour lines via Marching Squares algorithm
- Interactive 3D risk markers with hover popup and click info
- Rainfall data fetching from Meteostat API via RapidAPI
- Flood affected area calculator with real-world coordinate math

1.2 Technology Stack

Component	Technology	Purpose
UI Framework	Qt 6.x (C++)	Main application framework, widgets, signals
3D Rendering	OpenGL 3.3 Core / GLSL	Terrain mesh, shaders, billboard markers
GIS I/O	GDAL Library	GeoTIFF loading, coordinate transforms
Satellite Source	ESRI World Imagery	Tile server for HD satellite textures
Elevation Data	OpenTopography SRTMGL1	30m resolution DEM API
Flood Prediction	Custom ML DLL Engine	Physics + ML flood depth prediction
AI Alerts	Groq API / LLaMA 3.1 8B	Natural language flood warnings
Cloud Storage	Azure Blob Storage (SAS)	Persistent session data storage
Rainfall Data	Meteostat / RapidAPI	Historical precipitation data
Build System	CMake 3.16+	Cross-platform build configuration

1.3 Architecture Overview



2.1 Project Layout

```

Flood3D/
|-- mainwindow.h / mainwindow.cpp      # Core controller
|-- mapcanvas2d.h / mapcanvas2d.cpp    # 2D QGraphicsView
    map
|-- mapcanvas3d.h / mapcanvas3d.cpp    # 3D OpenGL widget
|-- floodengineclient.h / .cpp         # ML DLL interface
|-- dialog.cpp / rainfall_dialog.cpp   # Input dialogs
|-- vertex.vert                       # GLSL vertex
    shader
|-- fragment.frag                     # GLSL fragment
    shader
|-- shaders.qrc                       # Qt shader
    resource file
|-- CMakeLists.txt                   # Build
    configuration
|-- 3D/
    |-- headers/
    |   |-- terrainMesh.h
    |   |-- cameracontroller.h
    |   |-- satellitetexture.h
    |   |-- riskmarker.h
    |   |-- waterlayer.h
    |-- mains/
    |   |-- terrainMesh.cpp           # DEM mesh +
    |       smoothing
    |   |-- cameracontroller.cpp      # Mouse/keyboard
    |       camera
    |   |-- satellitetexture.cpp      # Tile fetch + GPU
    |       upload
    |   |-- riskmarker.cpp           # Billboard markers
    |-- waterlayer.cpp               # Water layer
        (unused)

```

2.2 Module Responsibilities

File	Module	Key Responsibilities
mainwindow.cpp	Core	Simulation flow, API calls, UI events, Azure upload
mapcanvas2d.cpp	2D View	Satellite tiles, flood overlay, contours, watershed
mapcanvas3d.cpp	3D View	OpenGL init, paintGL, camera, popup overlay
terrainMesh.cpp	3D	Vertex/index buffers, normals, smoothing, skirts
cameracontroller.cpp	3D	View matrix, mouse orbit, pan, zoom
satellitetexture.cpp	3D	Async tile fetch, stitch, QOpenGLTexture upload
riskmarker.cpp	3D	Billboard shaders, pulse animation, marker geometry
vertex.vert	Shader	Transform, UV, normals, world position
fragment.frag	Shader	Satellite, heatmap, lighting, fog, skirt walls
floodengineclient.cpp	Engine	DLL init, loadDEM, predictFlood, isReady

Completed Features

3.1 DEM Loading & Generation

▲ Digital Elevation Model

- ✓ Load from **GeoTIFF** via GDAL library with full metadata
- ✓ Load from **CSV** format for pre-processed data
- ✓ **Generate** by entering lat/lon bounding box – downloads SRTMGL1 30m data
- ✓ Automatic min/max elevation detection
- ✓ Gaussian smoothing for urban terrain (15 iterations)
- ✓ Saves CSV copy + uploads to Azure automatically

3.2 Satellite Imagery

✂ Multi-Tile Satellite System

Source: ESRI World Imagery (server.arcgisonline.com)

Parameter	2D Map	3D View	Notes
Default Zoom	16	17	Higher = sharper
Max Tiles	256	64	Auto zoom-out if exceeded
Tile Size	256x256 px	256x256 px	ESRI standard
Filtering	SmoothTransform	Aniso x16 + Mipmap	GPU quality

- Web Mercator tile math (XYZ tile system)
- Async fetch via `QNetworkAccessManager`
- `QPainter` stitching into single high-res image
- Full resolution display (no downscaling in 2D)

3.3 Flood Simulation Engine

🌊 ML Flood Prediction

- ✓ Custom ML DLL (`FloodEngine`) loaded at runtime
- ✓ Async execution via `QtConcurrent::run()` – no UI blocking
- ✓ Input: DEM grid + rainfall amount (mm)
- ✓ Output: 2D flood depth grid (mm per cell)
- ✓ Post-processing: Low / Medium / High risk classification
- ✓ Automatic statistics, Azure upload, AI report trigger

Risk Classification:

Risk Level	Threshold	Color	Marker
Low	norm < 0.3	Blue	—
Medium	0.3 – 0.6	Orange	⬮
High	> 0.6	Red	⬮

3.4 3D Terrain Rendering

OpenGL 3.3 Core Terrain

Feature	Implementation
Mesh Generation	Grid-based triangle strip, ~45,000+ vertices
Height Scaling	Adaptive based on elevation range
Smoothing	Gaussian filter, 3–15 iterations
Normals	Per-vertex central differencing
Terrain Skirts	Side walls + bottom face (dark brown earth)
Skybox	Full-screen quad, deep navy to blue-gray gradient
Lighting	Lambert diffuse + AO approximation + specular
Fog	Exponential atmospheric distance fade
Heatmap	Yellow-Orange-Red-Purple flood depth gradient

3.5 Risk Marker System

3D Billboard Markers

- ✓ Camera-facing billboard quads using view matrix right/up vectors
- ✓ Pin shape: circle top + triangle pointer (pure GLSL fragment shader)
- ✓ Pulse animation: 12% size oscillation via `sin(time*3.0)`
- ✓ Inner glow + white specular highlight on circle
- ✓ Click detection: MVP projection to screen, 20px radius
- ✓ QPainter popup overlay with risk info card
- ✓ Popup shows: Risk level, Row, Col, Flood Depth (m)
- ✓ Color-coded header bar matching risk level
- ✓ Arrow pointing down to marker position
- ✓ Right-click dismisses popup

3.6 2D Map Features

3.6.1 Flood Overlay

Semi-transparent color overlay scaled to full satellite resolution. Blue = shallow, Orange = medium, Red = severe flooding.

3.6.2 Contour Lines

Marching Squares algorithm draws elevation contour rings directly on satellite imagery with adaptive intervals and elevation labels.

Elev Range	Minor	Major	Typical Area
< 50m	2m	10m	Plains, coastal
50–200m	10m	50m	Rolling hills
200–500m	20m	100m	Moderate mountains
> 500m	50m	250m	High mountains

3.6.3 Watershed Detection

D8 flow direction algorithm identifies drainage basins. Flood-fill labeling from pour points. Color-coded regions with white boundary lines.

3.6.4 Cell Click

Click any cell to see: Row, Column, Elevation (m), Flood Depth (m) in styled console output.

3.7 AI-Powered Analysis

Groq LLaMA 3.1 Integration

Flood Alert (auto after simulation):

- Input: rainfall, max depth, risk cell counts
- Format: Risk level + Water depth + Recommended action
- Temperature: 0.3 for consistent output

Impact Report (auto after simulation):

- Flood severity distribution analysis
- Infrastructure stress assessment
- Mitigation action suggestions
- Grid-aware (no hallucinated population data)

3.8 Azure Cloud Integration

File	Azure Path	Content
dem.csv	runs/DATE/SESSION/	Elevation grid
flood_depth.csv	runs/DATE/SESSION/	Simulation output
metadata.json	runs/DATE/SESSION/	Parameters & stats
risk_map.png	runs/DATE/SESSION/	2D visualization
flood_alert.txt	runs/DATE/SESSION/	AI warning text
impact_report.txt	runs/DATE/SESSION/	AI impact analysis

3.9 Rainfall Data System

Meteostat API via RapidAPI – fetches daily precipitation for selected date range at DEM center coordinates. Aggregates to average monthly rainfall, applies 30-day accumulation for simulation.

3.10 Flood Area Calculator

Tools menu item calculates real-world flood coverage using geographic coordinate math. Outputs: Total Area (km²), Flooded Area (km²), Coverage percentage.

Environment Variables

Required API Keys & Secrets

Set these in Qt Creator: **Projects** → **Run** → **Environment**

Variable	Description
AZURE_BLOB_SAS	Full Azure SAS container URL with token
GROQ_API_KEY	Groq API key for LLaMA 3.1 access
RAPIDAPI_KEY	RapidAPI key for Meteostat rainfall data
OPENTOPO_API_KEY	OpenTopography API key for DEM downloads

Microsoft Store Release Checklist

5.1 What Needs to be Done

5.1.1 Technical Requirements

✂ Pre-Release Technical Tasks

- 🕒 **MSIX Packaging** – Package app using MSIX format for Store submission
- 🕒 **App Identity** – Set Publisher ID, Package Family Name in Package.appxmanifest
- 🕒 **Capability Declarations** – Declare: internetClient, privateNetworkClientServer
- 🕒 **Qt DLL Bundling** – Include all Qt6 DLLs, OpenGL, GDAL, FloodEngine.dll
- 🕒 **Visual C++ Redistributable** – Bundle VC++ runtime or mark as dependency
- 🕒 **App Icon** – Create icon set: 44x44, 150x150, 300x300, 620x300 px (PNG)
- 🕒 **Screenshots** – Min 1, max 10 screenshots at 1366x768 or higher
- 🕒 **Privacy Policy** – Required URL for any app using internet/APIs
- 🕒 **Age Rating** – Complete IARC questionnaire (likely Everyone rating)
- 🕒 **Windows 10/11 Testing** – Test on clean Windows install without dev tools

5.1.2 Core Features Still Missing

A Incomplete Features for Full Release

Feature	Priority	Status	Notes
Time-based flood simulation	HIGH	 In Progress	Hour by hour
Evacuation route suggestion	HIGH	 Planned	Dijkstra pathfinding
Vector data overlay (roads/buildings)	HIGH	 Planned	GeoJSON support
Early warning threshold system	MED	 Planned	User-set alert level
Multiple rainfall scenario compare	MED	 Planned	3 scenarios side-by-side
Historical flood data comparison	MED	 Planned	Compare past events
Population risk assessment	MED	 Planned	Census data overlay
Smooth camera damping	LOW	 Planned	Inertia on camera
Water flow animation	LOW	 Planned	Visual flow lines
Multi-area comparison	LOW	 Planned	Side-by-side areas

5.1.3 Polish & UX Tasks

-  Add loading spinner during DEM download and tile fetch
-  Error dialogs instead of console log for user-facing errors
-  Settings panel for API key management (instead of env vars)
-  Onboarding tutorial / welcome screen for new users
-  Export simulation report as PDF from within the app
-  Keyboard shortcuts documentation
-  App crash recovery / autosave session state

5.2 Recommended Release Timeline



Data Flow Diagram

3D heatmap; [step] (s8) at (12,-3) Risk markers
generated; [result] (s9) at (4,-6) Groq AI
alert + report; [result] (s10) at (8,-6) Azure Blob
all files uploaded;
[arr] (s1)–(s2); [arr] (s2)–(s3); [arr] (s3)–(s4); [arr] (s3)–(s6); [arr] (s5)–(s6); [arr]
(s6)–(s7); [arr] (s6)–(s8); [arr] (s7)–(s9); [arr] (s7)–(s10); [arr] (s8)–(s10); [arr] (s9)–(s10);
[note] at (2,0.5) Step 1; [note] at (6,0.5) Step 2; [note] at (10,0.5) Step 3;

Future Roadmap

7.1 Time-Based Flood Simulation (Next Feature)

🕒 Hour-by-Hour Flood Progression

Phase 1 – Engine: Post-process final flood grid into N time steps using realistic growth curve (10% → 25% → 45% → 65% → 80% → 100%)

Phase 2 – Data: Store vector of flood grids in memory, one per time step

Phase 3 – UI Controls:

[|<<] [|<] [|/] [|>] [||>] Speed: [1x] Hour: [3/12]

Phase 4 – Visualization: 2D + 3D both animate per frame simultaneously

Phase 5 – Timeline Panel: Hour-by-hour stats in Timeline tab

7.2 Complete Roadmap

Feature	Priority	Value
Time-based simulation	HIGH	Most visually impressive
Evacuation route suggestion	HIGH	Real humanitarian value
Vector overlay (roads/buildings)	HIGH	Infrastructure risk context
Early warning threshold	MED	Disaster management utility
Multi-scenario comparison	MED	Decision support tool
Historical data comparison	MED	Scientific validation
Population risk assessment	MED	Impact quantification
Smooth camera damping	LOW	UX polish
Water flow animation	LOW	Visual quality
Multi-area comparison	LOW	Research tool

FloodSim GIS

Built with Qt6, OpenGL, GDAL, Azure & AI

Documentation generated: March 2026

Version 1.0 – Pre-Release

